МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Вариант 3

Студент гр. 4343 _____ Едина А.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Задание

1 задание.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст (T,1≤|T|≤100000T,1≤|T|≤100000).

Вторая – число n(1≤n≤3000), каждая следующая из n строк содержит шаблон из набора P={p1,…,pn}1≤|pi|≤75

Все строки содержат символы из алфавита {A,C,G,T,N}

Выход:

Все вхождения образцов из P в T.

Каждое вхождение образца в текст представить в виде двух чисел – i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

2 задание.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T.

Например, образец ab??c? с джокером ? встречается дважды в тексте xabvccbababcax.

Символ джокер не входит в алфавит, символы которого используются в T. Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида ??? недопустимы.

Все строки содержат символы из алфавита {A,C,G,T,N}

Вход:

Текст (T, $1 \le |T| \le 100000$)

Шаблон (P, $1 \le |P| \le 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Индивидуальное задания.

Вариант 3. Вычислить длину самой длинной цепочки из суффиксных ссылок и самой длинной цепочки из конечных ссылок в автомате.

Выполнение работы.

1 задание

Файл main1.py

Программа считывает строку text, затем целое число n и n строк-паттернов.

Строится бор (префиксное дерево) из всех паттернов. Каждая вершина содержит:

- next — переходы по символам алфавита {A, C, G, T, N} в боре,
- go — переходы в автомате (с учётом суффиксных ссылок),
- sufflink — суффиксная ссылка,
- dict_link — сжатая суффиксная ссылка (на ближайшую терминальную вершину),
- pattern_ids — список идентификаторов паттернов, заканчивающихся в данной вершине.

После построения бора выполняется обход в ширину для расстановки суффиксных ссылок и вычисления переходов go.

Затем происходит проход по тексту: для каждого символа выполняется переход по go и обход по сжатым ссылкам для обнаружения всех вхождений паттернов.

При нахождении паттерна в результат добавляется пара (позиция начала вхождения, id паттерна).

Результаты сортируются по позиции и выводятся.

2 задание

Файл main2.py

Программа решает задачу поиска шаблона p, содержащего символ-джокер (любой символ, который может быть произвольным символом текста).

Сначала функция parser(p, joker) разбивает шаблон на «куски» — непрерывные последовательности символов, не включающие джокера. Каждый кусок представляется как (смещение от начала шаблона, подстрока).

Далее строится автомат Ахо-Корасик для всех таких кусков. В терминальных вершинах хранятся parts — список пар (смещение, подстрока).

При проходе по тексту для каждой позиции i обновляется массив counts, где counts[start_pos] — количество найденных кусков, соответствующих шаблону в позиции start_pos.

Если counts[i] == количество кусков — значит, все части шаблона совпали с учётом джокеров, и позиция i+1 выводится как начало вхождения.

Индивидуальное задание

Файл main.py

Программа расширяет main1.py.

При построении бора каждая вершина получает уникальный id и символ, по которому в неё пришли (road_char).

Функция chain вычисляет максимальную длину цепочки суффиксных ссылок и цепочки сжатых ссылок среди всех вершин.

Функция print_debug_info выводит для каждой вершины:

- её идентификатор,
- символ перехода,
- номер вершины, на которую ведёт суффиксная ссылка,
- номер вершины, на которую ведёт сжатая ссылка,
- список идентификаторов паттернов (если вершина терминальная).

В процессе поиска в тексте дополнительно выводятся переходы между вершинами и факт обнаружения каждого паттерна.

Исследование

Задание 1

n – длина входного текста (text).

k – количество паттернов (patterns).

L – суммарная длина всех паттернов (sum(len(p) for p in patterns)).

|Σ| – размер алфавита (в данном случае фиксирован: 5, символы ACGTN).

m – общее количество найденных вхождений паттернов в тексте.

Пространственная сложность

Хранение бора

Каждый узел Vertex хранит два списка (next и go) размером |Σ|. Количество узлов в худшем случае равно L + 1 (корень + по одному узлу на каждый символ всех паттернов, если они не пересекаются).

Сложность: $O(L * |\Sigma|)$. При |Σ| = 5 это $O(L)$.

Хранение паттернов и результатов

Список patterns занимает $O(L)$.

Список results хранит все найденные вхождения, что занимает $O(m)$.

Итоговая пространственная сложность: $O(L * |\Sigma| + m)$.

Временная сложность

Построение бора

Мы проходим по каждому символу каждого паттерна один раз.

Сложность: $O(L * |\Sigma|)$. Поскольку |Σ| = 5 – это константа, можно считать как $O(L)$.

Построение ссылок

Алгоритм обходит каждое состояние (узел) в боре. Количество узлов не превышает L. Для каждого узла проверяются переходы по всем символам алфавита.

Сложность: $O(L * |\Sigma|)$ или просто $O(L)$.

Поиск в тексте

Переходы по состояниям: Благодаря предварительному вычислению v.go[c], переход к следующему состоянию по символу текста занимает $O(1)$. Для всего текста это $O(n)$.

Обход сжатых суффиксных ссылок (dict_link): В каждой позиции текста мы можем найти несколько вхождений. Благодаря сжатым ссылкам, мы посещаем только те узлы, которые действительно являются терминальными (концами паттернов). Это занимает $O(m)$ времени за весь процесс.

Сложность: $O(n + m)$.

Итоговая временная сложность: $O(L + n + m)$.

Задание 2

n – длина входного текста (text).

M — длина исходного паттерна с джокерами (p).

k – количество паттернов (patterns).

L – суммарная длина всех фрагментов (за вычетом джокеров, $L \leq M$).

|Σ| – размер алфавита (в данном случае фиксирован: 5, символы ACGTN).

m – общее количество найденных вхождений паттернов в тексте.

K – количество подстрок (фрагментов), на которые разбился паттерн.

O – общее количество вхождений всех фрагментов в тексте.

Пространственная сложность

Хранение автомата

Сложность: $O(L * |\Sigma|)$.

Массив счетчиков (counts)

Создается массив размером n+1 для хранения количества найденных частей для каждой возможной начальной позиции.

Сложность: $O(n)$.

Хранение фрагментов

Список parts хранит подстроки и их смещения.

Сложность: $O(M)$.

Итоговая пространственная сложность: $O(n + M + L * |\Sigma|)$.

Временная сложность

Парсинг паттерна

Проход по строке паттерна один раз для выделения фрагментов и их смещений (offset).

Сложность: $O(M)$.

Построение автомата

Аналогично первой программе: построение бора, суффиксных и сжатых ссылок для K фрагментов.

Сложность: $O(L * |\Sigma|)$, что при константном алфавите сводится к $O(L)$.

Поиск фрагментов в тексте

Проход по тексту: $O(n)$ итераций, в каждой из которых переход по состоянию занимает $O(1)$.

Обход сжатых ссылок и обновление counts: Для каждого найденного фрагмента выполняется обновление массива счетчиков. Суммарно это занимает время, пропорциональное количеству вхождений фрагментов.

Сложность: $O(n + O)$.

Финальная проверка массива counts:

Проход по массиву длиной n для поиска позиций, где количество совпавших фрагментов равно их общему числу K.

Сложность: $O(n)$.

Итоговая временная сложность: $O(M + L + n + O)$.

$L \leq M$ => $O(M + n + O)$.

Индивидуальное задание

N – длина текста.

L – суммарная длина всех паттернов (sum(len(p) for p in patterns)).

H – максимальная длина одного паттерна (высота бора).

|Σ| – размер алфавита (константа 5).

m – общее количество найденных вхождений в тексте.

Пространственная сложность

Хранение автомата

Сложность: $O(L * |\Sigma|)$.

Список узлов (nodes)

Итого: $O(L)$.

Хранение результатов и паттернов

Паттерны: O(L).

Найденные вхождения в results: O(m).

Суммарная пространственная сложность: $O(L * |\Sigma| + m)$

Временная сложность

Построение бора и ссылок (build)

Сложность: $O(L * |\Sigma|)$.

Анализ цепочек (chain):

Эта функция проходит по каждому узлу бора ($O(L)$).

Для каждого узла она запускает циклы while, которые поднимаются до корня по суффиксным и сжатым ссылкам. В худшем случае (например, паттерны "A", "AA", "AAA"...) длина такой цепочки пропорциональна H.

Итого для этапа: $O(L * H)$.

Поиск в тексте:

Итого: $O(N + m)$.

Суммарная временная сложность: $O(L * |\Sigma| + L * H + N + m)$

Тестирование

Таблица 1 - Результаты тестирование для задания 1

Тест	Входные данные	Выходные данные
1	AAAAA 3 A AA AAA	1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3 4 1 4 2 5 1
2	AGTAGT 1 C	
3	ACGTT 2 AC TT	1 1 4 2
4	ATATA 1 ATA	1 1 3 1
5	A 1 A	1 1

Таблица 2 - Результаты тестирования для задания 2

Тест	Входные данные	Выходные данные
1	ATAGA A?A ?	1 3
2	ACGGGT A??G ?	1
3	TAGA TA?? ?	1
4	GATAG ??AG ?	2
5	AGAGAG AG?AG ?	

Таблица 3 - Результаты тестирования для индивидуального задания

Тест	Входные данные	Выходные данные

1	AAAAA 3 A AA AAA	максимальная длина цепочки суффиксных ссылок: 3 максимальная длина цепочки сжатых ссылок: 2
2	AAGTA 2 ACGT GT	максимальная длина цепочки суффиксных ссылок: 2 максимальная длина цепочки сжатых ссылок: 1
3	ACTTCG 2 ACT TCG	максимальная длина цепочки суффиксных ссылок: 1 максимальная длина цепочки сжатых ссылок: 0
4	ACGTACGTGACG 2 AC ACGT	максимальная длина цепочки суффиксных ссылок: 1 максимальная длина цепочки сжатых ссылок: 0
5	NTAG 2 NT NG	максимальная длина цепочки суффиксных ссылок: 1 максимальная длина цепочки сжатых ссылок: 0

Приложение

Код из main1.py:

```
class Vertex:
    def __init__(self, alpha=5):
        self.next = [None] * alpha  # список переходов в боре
        self.go = [None] * alpha  # список переходов в автомате
        self.sufflink = None
        self.dict_link = None
        self.pattern_ids = []

def num(c):
    return "ACGTN".index(c)

def build(patterns):
    root = Vertex()

    for idx, p in enumerate(patterns, start=1):
        v = root
        for i in range(len(p)):
            if v.next[num(p[i])] is None:
                v.next[num(p[i])] = Vertex()
            v = v.next[num(p[i])]
        v.pattern_ids.append(idx)

    from collections import deque
    queue = deque([root])
    root.sufflink = root
    root.dict_link = root

    while queue:
        v = queue.popleft()

        for c in range(5):
            child = v.next[c]
            if child:
                # суффиксная ссылка для ребенка
                if v == root:
                    child.sufflink = root
                else:
                    child.sufflink = v.sufflink.go[c]

                # ближайший терминальный узел в цепочке суффиксов
                if child.sufflink.pattern_ids:
                    child.dict_link = child.sufflink
                else:
                    child.dict_link = child.sufflink.dict_link

                child.go[c] = child
                queue.append(child)

            if child:
                v.go[c] = child
            else:
                if v == root:
                    v.go[c] = root
                else:
```

```
                        v.go[c] = v.sufflink.go[c]
    return root

text = input().strip()
n = int(input())

patterns = []
for _ in range(n):
    patterns.append(input().strip())

results = []
root = build(patterns)

v = root
for i, char in enumerate(text):
    v = v.go[num(char)]

    # по терминальным узлам через сжатые ссылки
    temp = v
    while temp != root:
        if temp.pattern_ids:
            for p_id in temp.pattern_ids:
                results.append((i - len(patterns[p_id-1]) + 2,
p_id))
        temp = temp.dict_link

results.sort()
for pos, p_id in results:
    print(pos, p_id)
```

Код из main2.py:

```
class Vertex:
    def __init__(self, alpha=5):
        self.next = [None] * alpha  # список переходов в боре
        self.go = [None] * alpha  # список переходов в автомате
        self.sufflink = None
        self.dict_link = None
        self.parts = []

def num(c):
    return "ACGTN".index(c)

def parser(p, joker):
    start = -1
    cur_sub = ""
    parts = []
    for i, char in enumerate(p):
        if char != joker:
            if start == -1:
                start = i
            cur_sub += char
        else:
            if cur_sub:
                parts.append((start, cur_sub))
                cur_sub = ""
```

```
                start = -1
    if cur_sub:
        parts.append((start, cur_sub))
    return parts

def build(parts):
    root = Vertex()

    for offset, sub in parts:
        v = root
        for char in sub:
            c = num(char)
            if v.next[c] is None:
                v.next[c] = Vertex()
            v = v.next[c]

        v.parts.append((offset, sub))

    from collections import deque
    queue = deque([root])
    root.sufflink = root
    root.dict_link = root

    while queue:
        v = queue.popleft()

        for c in range(5):
            child = v.next[c]
            if child:
                # суффиксная ссылка для ребенка
                if v == root:
                    child.sufflink = root
                else:
                    child.sufflink = v.sufflink.go[c]

                # ближайший терминальный узел в цепочке суффиксов
                if child.sufflink.parts:
                    child.dict_link = child.sufflink
                else:
                    child.dict_link = child.sufflink.dict_link

                child.go[c] = child
                queue.append(child)

            if child:
                v.go[c] = child
            else:
                if v == root:
                    v.go[c] = root
                else:
                    v.go[c] = v.sufflink.go[c]
    return root

text = input().strip()
p = input().strip()
joker = input().strip()

parts = parser(p, joker)
```

```
root = build(parts)
counts = [0] * (len(text)+1)

v = root
for i, char in enumerate(text):
    char_idx = num(char)
    if char_idx == -1:
        v = root
        continue

    v = v.go[num(char)]

    temp = v
    while temp != root:
        for offset in temp.parts:
            start_pos = i - len(offset[1]) + 1 - offset[0]
            if start_pos >= 0 and start_pos <= len(text) - len(p):
                counts[start_pos] += 1
        temp = temp.dict_link

num_parts = len(parts)
for i in range(len(text)):
    if counts[i] == num_parts:
        print(i + 1)
```

Код main.py:

```
class Vertex:
    def __init__(self, id, road_char, alpha=5):
        self.id = id
        self.road_char = road_char
        self.next = [None] * alpha  # список переходов в боре
        self.go = [None] * alpha  # список переходов в автомате
        self.sufflink = None
        self.dict_link = None
        self.pattern_ids = []

def num(c):
    return "ACGTN".index(c)

def build(patterns):
    nodes = []
    root = Vertex(id=0, road_char="root")
    counter = 1

    for idx, p in enumerate(patterns, start=1):
        v = root
        for i in p:
            c = num(i)

            if v.next[c] is None:
                v.next[c] = Vertex(id=counter, road_char=i)
                nodes.append(v.next[c])
                counter += 1

            v = v.next[c]
```

```
            v.pattern_ids.append(idx)

        from collections import deque
        queue = deque([root])
        root.sufflink = root
        root.dict_link = root

        while queue:
            v = queue.popleft()

            for c in range(5):
                child = v.next[c]
                if child:
                    # суффиксная ссылка для ребенка
                    if v == root:
                        child.sufflink = root
                    else:
                        child.sufflink = v.sufflink.go[c]

                    # ближайший терминальный узел в цепочке суффиксов
                    if child.sufflink.pattern_ids:
                        child.dict_link = child.sufflink
                    else:
                        child.dict_link = child.sufflink.dict_link

                    child.go[c] = child
                    queue.append(child)

                if child:
                    v.go[c] = child
                else:
                    if v == root:
                        v.go[c] = root
                    else:
                        v.go[c] = v.sufflink.go[c]
        return root, nodes

    def chain(all, root):
        max_suff = 0
        max_dict = 0

        for v in all:
            # суффиксная цепочка
            curr_suff = 0
            temp = v
            while temp != root:
                curr_suff += 1
                temp = temp.sufflink
            max_suff = max(max_suff, curr_suff)

            # цепочка конечных ссылок
            curr_dict = 0
            temp = v
            while temp != root and temp.dict_link != root:
                curr_dict += 1
                temp = temp.dict_link
            max_dict = max(max_dict, curr_dict)
```

```
        return max_suff, max_dict

    def print_debug_info(all_nodes):
        print()
        for v in all_nodes:
            s = v.sufflink.id if v.sufflink else 0
            d = v.dict_link.id if v.dict_link else 0
            term = f" (шаблоны: {v.pattern_ids})" if v.pattern_ids else
""
            print(f"узел {v.id} ['{v.road_char}']: суффисная ссылка на
узел -> {s}, сжатая ссылка на узел -> {d}{term}")
        print()

    if __name__ == "__main__":
        # text = input().strip()
        # n = int(input())

        # patterns = []
        # for _ in range(n):
        #     patterns.append(input().strip())
        text = "NTAG"
        patterns = ["NT", "NG"]

        results = []
        root, nodes = build(patterns)

        print(f"текст: {text}")
        print(f"список паттернов: {patterns}\n")

        print("-"*40)
        print("бор:")
        print_debug_info(nodes)

        max_suff, max_dict = chain(nodes, root)
            print(f"максимальная длина цепочки суффиксных ссылок:
{max_suff}")
        print(f"максимальная длина цепочки сжатых ссылок: {max_dict}")

        print()
        print("-"*40)
        print("ищем шаблоны в тексте")
        v = root
        for i, char in enumerate(text):
            old = v.id
            v = v.go[num(char)]
            print(f"[{i+1}] символ '{char}': узел {old} -> узел {v.id}")

            # по терминальным узлам через сжатые ссылки
            temp = v
            while temp != root:
                if temp.pattern_ids:
                    for p_id in temp.pattern_ids:
                        results.append((i - len(patterns[p_id-1]) + 2,
p_id))
                                print(f"найдено совпадение шаблона
'{patterns[p_id-1]}' (id={p_id}) в позиции {i - len(patterns[p_id-1]) +
2}")
                temp = temp.dict_link
```

```
results.sort()
for pos, p_id in results:
    print(pos, p_id)
```